

Reconstruct, Verify, Ship

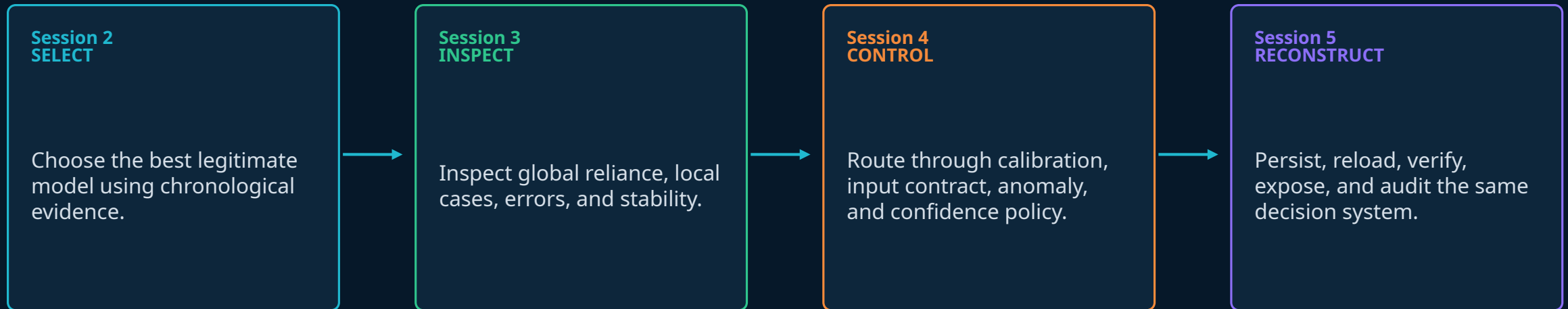
AI Productization, Final Demo, and Deployment

The Session 3 + Session 4 decision contract must rebuild from persisted artifacts and run outside notebook memory.

The exact Session 3 + Session 4 decision contract must rebuild from persisted artifacts in a fresh process.

Selection -> Inspection -> Control -> Reconstruction

Session 5 does not invent a new model or a new policy. It productizes the evidence chain already defended.



Final question: Can the same controlled behavior survive outside notebook memory?

What must be proven before the model is shipped

The goal is reproducible behavior: recover, persist, verify, expose, and deploy the already-defended decision path.

Recover

Exact selected model - no hard-coding

Freeze

Calibration, thresholds, feature order, versions

Verify

Save/load parity + artifact fingerprints

Rebuild

Cold-start process with no notebook variables

Route

All four Session 4 states from loaded artifacts

Expose

Thin app + regression tests + review JSON

Primary artifact: Reliable AI Risk Intelligence Application

The notebook works. Now kill it.

Two screens can show the same prediction while only one is a reproducible product.

DEMO A - WARM NOTEBOOK

- Model already in memory
- Thresholds live in variables
- Feature order is implicit
- Restart destroys state
- No independent reconstruction proof

DEMO B - COLD-START PRODUCT

- Artifacts loaded from disk
- Metadata declares feature + policy identity
- Same request reproduces same route
- Regression checks run independently
- Review evidence is serializable

Prediction checkpoint: Which evidence distinguishes product B from a lucky demo?

Productization vocabulary used today

Each term has a concrete engineering consequence in the notebook.

Artifact

Saved object/evidence needed to reproduce behavior

Bundle

Model + policy + metadata + evidence files

Metadata contract

Declared product identity and feature/policy order

Cold start

New process with no inherited notebook state

Training-serving skew

Training and application build different inputs

Regression test

Executable claim protecting known behavior

Thin UI

Interface delegates to shared decision logic

Audit record

Serializable why/status/threshold/version evidence

Fingerprint

Digest that detects byte-level artifact changes

Session 5 starts from fixed checkpoints

These are reference checkpoints from the refined Sessions 3 and 4 - not values tuned in Session 5.

840

final rows

logistic

selected model

0.6357

2025 Macro F1

1.0883

temperature T

0.650

review threshold

-0.0404

anomaly threshold

70.83%

2025 coverage

29.17%

review rate

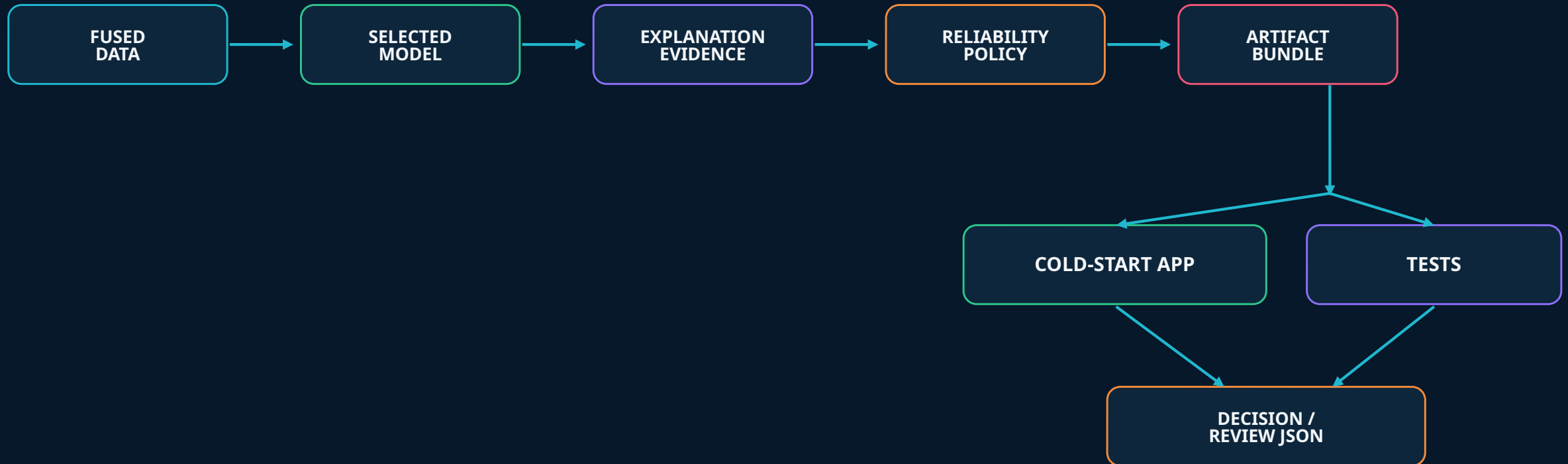
80.00%

selective accuracy

Guardrail: 2025 is reported here; it is not reopened to choose a different model or threshold.

One decision contract from data to audit evidence

The interface and the tests consume the same persisted product path.



Module responsibilities after refactoring

Earlier sessions keep logic visible. Session 5 consolidates understood logic into reusable modules.

MODULE	RESPONSIBILITY
constants.py	keys, feature lists, class order, valid ranges
data.py	audit, cleaning, fusion
features.py	feature engineering + one-request schema
modeling.py	chronological split, comparison, selection
explainability.py	permutation, local occlusion, Tree SHAP
reliability.py	calibration, threshold policy, validation, anomaly, routing
artifacts.py	persist/load exact selected product path
app.py	thin input/output adapter around shared decision function

A model file alone is not the product

Every artifact below changes or documents the behavior of the decision system.

risk_model.joblib

exact legitimate classifier

temperature_scaler.joblib

confidence calibration

anomaly_model.joblib

valid-but-unusual review

reference_medians.joblib

local explanation reference

metadata.json

feature + policy identity

model_comparison.csv

selection evidence

threshold_analysis.csv

policy trade-off evidence

artifact_manifest.json

SHA-256 fingerprints

Product identity = parameters + schema + policy + evidence context.

Make product identity explicit

The next process should not guess feature order, class order, years, or policy values.

MODEL + SCHEMA IDENTITY

```
selected_model_name  
selected_model_class  
model_features  
monitor_features  
class_names  
random_state
```

EVIDENCE + POLICY IDENTITY

```
training_years  
selection_year = 2023  
calibration_year = 2024  
test_year = 2025  
temperature = 1.0883  
review_threshold = 0.650  
anomaly_threshold  $\approx$  -0.0404  
policy_version
```

Training-serving skew can silently change the system

A correct model with a wrong feature builder or feature order is still an incorrect product.

TRAINING PATH

```
raw measurements  
-> engineer_features()  
-> MODEL_FEATURES exact order  
-> fitted pipeline
```

BROKEN SERVING PATH

```
raw measurements  
-> duplicated formulas / reordered  
fields  
-> "same" model file  
-> different effective decision system
```

Rule: one shared feature builder, one declared MODEL_FEATURES order.

Recover the evidence-selected model - do not re-shop

Productization consumes the selection record returned by `compare_candidates()`.

INVALID TEMPTATION

advanced_leaky
Macro F1 ≈ 0.9654

High score - but prohibited post-outcome feature access makes it ineligible.

SELECTED PATH

logistic
Macro F1 ≈ 0.6357

Legitimate chronological path; persisted as the product model.

SELECTION RULE

Eligibility first
Then Macro F1
Then tie-break rules

The winner is read from the comparison record.

A large score never overrides feature eligibility.

Save the exact selected model - not a favorite model name

The persistence call should receive the object returned by the evidence-based selection path.

REFERENCE PATTERN

```
selected_model_name = comparison["selected_model_name"]
selected_model = comparison["selected_model"]

joblib.dump(selected_model, MODEL_DIR / "risk_model.joblib")
metadata["selected_model_name"] = selected_model_name
metadata["model_features"] = list(MODEL_FEATURES)
```

The model object is persisted because it won the declared legitimate selection rule.

Session 4 reliability stays fixed

Session 5 reconstructs the control layer; it does not tune a new one from 2025.

CALIBRATION

Temperature
 $T = 1.0883$

Fitted on 2024 calibration evidence.

CONFIDENCE POLICY

Review threshold
 $\tau = 0.650$

Chosen before reading 2025 as a tuning target.

ANOMALY POLICY

2% calibration tail
 $\theta \approx -0.0404$

Valid-but-unusual cases are deferred.

Same classifier + different policy values = different decision system.

Bundle fingerprint - detect accidental artifact replacement

SHA-256 identifies bytes. It does not certify model quality.

MANIFEST ENTRY

```
risk_model.joblib  
bytes: 4,378  
sha256: 9f3b...a21c
```

```
metadata.json  
bytes: ...  
sha256: ...
```

WHAT IT PROVES

The file currently loaded
matches the file
fingerprinted when the
manifest was generated.

WHAT IT DOES NOT PROVE

Good accuracy
Safe policy
No bias
No drift
No maliciously generated original
bundle

Integrity evidence answers “which artifact?” - not “is the artifact scientifically correct?”

Save/load parity is the first executable product test

Persistence must not change the selected model's checked behavior.

IN-MEMORY

```
pred_original =  
selected_model.predict(X_probe)  
  
model = selected by comparison  
feature order = MODEL_FEATURES
```

RELOADED

```
loaded = joblib.load("risk_model.joblib")  
pred_loaded = loaded.predict(X_probe)  
  
assert_array_equal(pred_original,  
pred_loaded)
```

Parity proves serialization consistency for the checked cases - not future performance.

Failure 1 - hard-code a model because its name looks familiar

Productization must follow the selection record, not developer memory.

WRONG

```
product_model = comparison["models"]  
    ["advanced_correct"][0]  
  
# bypasses the evidence-selected winner
```

CORRECT

```
product_model =  
comparison["selected_model"]  
product_model_name =  
comparison["selected_model_name"]  
  
# persists the actual legitimate winner
```

The selection record is an input to productization.

Failure 2 - keep the model, corrupt the contract

A reversed feature list should be rejected before deployment.

ORIGINAL

```
temperature_c  
rainfall_mm  
humidity_pct  
...  
season_sin  
season_cos
```

DRIFTED

```
season_cos  
season_sin  
...  
humidity_pct  
rainfall_mm  
temperature_c
```

VALIDATOR

```
model_feature_order_mismatch  
  
RELEASE BLOCKED
```

"Same model file" is not enough when the input contract changed.

Metadata + manifest turn hidden assumptions into checks

Restore the contract from disk, then recalculate every file fingerprint.

- Reload metadata.json instead of reconstructing feature order from memory.
- Compare selected model name, class order, feature order, review threshold, and anomaly threshold.
- Recalculate SHA-256 for each artifact in artifact_manifest.json.
- Block the release when any mismatch is found.

A product contract should fail loudly when identity is inconsistent.

No notebook variable may be required

The subprocess receives only paths and source code.

1. START NEW PROCESS

```
python -c ...
```

```
No inherited model object  
No inherited thresholds
```

2. LOAD BUNDLE

```
load_artifacts(model_dir)
```

```
model  
calibrator  
anomaly model  
thresholds  
metadata
```

3. ROUTE REQUEST

```
make_safe_decision(...)
```

```
Expected:  
FRESH_PROCESS_MODEL=logistic  
FRESH_PROCESS_STATUS=ACCEPT  
_AI_DECISION
```

Cold-start success removes hidden notebook state from the evidence chain.

All four states must survive persistence

The first true condition still decides the route after reload.

INVALID

REJECT_INVALID_INPUT

hard contract fails

ANOMALOUS

HUMAN_REVIEW_ANOMALY

valid but unfamiliar

LOW CONF.

HUMAN_REVIEW_LOW_CONFIDENCE

valid + familiar, confidence < 0.650

PASS

ACCEPT_AI_DECISION

valid + familiar + confident

Persistence is correct only if the operational state machine is preserved.

Reference cases from Session 4

The same saved artifacts should reproduce these deterministic statuses.

CASE	CONFIDENCE	ANOMALY SCORE	EXPECTED STATUS
ACCEPT_CASE	0.9978	-0.0173	ACCEPT_AI_DECISION
LOW_CONFIDENCE_CASE	0.5548	0.0049	HUMAN_REVIEW_LOW_CONFIDENCE
ANOMALY_CASE	-	-0.0781	HUMAN_REVIEW_ANOMALY
IMPOSSIBLE_CASE	-	-	REJECT_INVALID_INPUT

Reject $\neq$ low confidence. Anomaly $\neq$ wrong. Accept $\neq$ certainty.

One reusable decision function under the UI

The interface collects values; the shared package owns validation, features, model, and route policy.



The same `make_safe_decision()` path is used by the notebook, app, and tests.

THIN ADAPTER RULE

UI code may format inputs/outputs; it must not fit models, duplicate feature formulas, or implement a second route state machine.

A beautiful interface is not a second AI system

src/app.py should load artifacts and delegate routing.

ALLOWED IN UI

- input controls
- loading / error states
- result cards
- formatting
- calling `make_safe_decision()`

NOT ALLOWED IN UI

- duplicate feature formulas
- hard-coded thresholds
- new model selection
- second anomaly rule
- second confidence router

One source of truth makes the interface auditable.

Human-review decisions should be serializable

A queue item should explain why review was requested without rerunning the whole notebook.

EXAMPLE REVIEW RECORD

```
{
  "status": "HUMAN_REVIEW_LOW_CONFIDENCE",
  "prediction": "LOW",
  "confidence": 0.5548,
  "review_threshold": 0.650,
  "anomaly_score": 0.0049,
  "anomaly_threshold": -0.0404,
  "model_version": "logistic",
  "policy_version":
"reliability_policy_session4_final",
  "top_factors": [...]
}
```

EXPORT RULE

Only HUMAN_REVIEW_* outcomes enter this review-record exporter.

VERIFY AFTER WRITE

Read the JSON back, confirm status and required fields, then keep it as evidence.

Regression tests protect the evidence-selected path

Tests should fail when product identity or safety behavior drifts.

fusion

840 rows, no duplicate keys, no missing model cells

time roles

training < calibration < final test

leakage

advanced_leaky cannot become selected product

probabilities

calibrated rows remain valid distributions

persistence

metadata and persisted model agree

rejection

invalid input returns REJECT_INVALID_INPUT

routing

declared cases reproduce expected states

app import

cold-start module import succeeds

A green suite proves declared contracts - not causality, fairness, or permanent future accuracy.

Persistence must match the selection record

The test protects against accidentally saving a different fitted model.

REFERENCE TEST

```
saved = load_artifacts(MODEL_DIR)
metadata = saved["metadata"]

assert metadata["selected_model_name"] == expected_selected_name
assert metadata["model_features"] == list(MODEL_FEATURES)
np.testing.assert_array_equal(
    selected_model.predict(X_probe),
    saved["model"].predict(X_probe),
)
```

This test would catch a hard-coded or stale persisted model path.

Green tests are necessary - and limited

Product QA should be precise about what software verification can and cannot establish.

GREEN TESTS CAN SUPPORT

- reproducible loading
- feature/order contract
- known route behavior
- invalid-input refusal
- selected-model persistence
- application importability

GREEN TESTS CANNOT GUARANTEE

- future distribution stability
- scientific causality
- absence of bias
- perfect calibration forever
- correctness on every unseen request
- security beyond tested boundaries

Use exact claims: "the contract passed" is stronger than "the AI is safe."

Evidence before spectacle

A short final demo should reconstruct the reasoning chain, not only click the UI.

- 01 **DATA CONTRACT** 840 rows + approved features
- 02 **MODEL SELECTION** leaky score rejected; logistic selected
- 03 **EXPLANATION** one case + one limitation
- 04 **RELIABILITY** four route states
- 05 **COLD START** reload bundle + manifest check
- 06 **APP + TESTS** thin UI + regression suite + review JSON

The interface is shown after the evidence chain - not instead of it.

Eight questions that connect evidence to engineering

Each answer is grounded in the same source package and persisted product contract.

1

Why is the 0.965-class leaky model invalid despite its score?

2

Why does changing feature order create a different effective system?

3

Why is $T = 1.0883$ stored with the artifact bundle?

4

Why is anomaly review different from invalid-input rejection?

5

What does a SHA-256 fingerprint prove - and not prove?

6

Which regression test catches a stale persisted model?

7

Why must the UI call one shared decision function?

8

What future drift can still break a fully green release?

LIVE HANDOFF

PDF -> Collaborative Notebook -> Verified App

The explanation is introduced first; the evidence is then rebuilt, broken, verified, and shipped in code.

PDF BRIDGE

connect Sessions 1-4 to the final product

PREDICT

lock a decision before code reveals the answer

REBUILD

recover the selected model and fixed policy from src

BREAK

sabotage a copied metadata contract safely

ROUTE

reproduce all four Session 4 decision states

VERIFY

manifest + regression tests + cold-start process

APP

open the thin Gradio interface from shared logic

DEPLOY

local, temporary public link, or hosted Space

DEFEND

show evidence, limitation, and one live change

DEPLOYMENT OPTIONS

From laptop to internet without changing model logic

The same `src/app.py` and `reliable_ai` package are used in every path.

LOCAL

```
python app.py
```

Fastest private demonstration.

TEMP PUBLIC

```
python app.py --share
```

Short external Gradio link.

HOSTED

Hugging Face Gradio Space

Persistent public demo when free CPU quota is available.

VERIFY FIRST

```
python verify_product.py
```

Artifact contract + four routes.

INSTALL

```
pip install -r requirements.txt
```

REBUILD

```
python train_model.py
```

Regenerate one matched bundle.

TEST

```
pytest -q
```

Protect known contracts.

ONE SOURCE

UI delegates to `predict_request()` and `make_safe_decision()`.

Public hosting changes accessibility - it does not increase scientific validity.

SHIP CHECKLIST

What must be true before a public link is shown

Every line below has executable or persisted evidence in the final package.

- The 840-row data contract and 16-feature model schema remain unchanged.
- The leaky candidate remains ineligible even when its score is higher.
- logistic, $T \approx 1.0883$, review threshold 0.650, and anomaly threshold ≈ -0.0404 travel together.
- artifact_manifest.json matches the current persisted bundle.
- ACCEPT / LOW-CONFIDENCE REVIEW / ANOMALY REVIEW / REJECT are reproduced after loading.
- python verify_product.py passes in a fresh process.
- pytest -q passes the expanded Session 5 regression contract.
- python app.py --smoke-only passes before a browser or public link is opened.
- requirements.txt and DEPLOYMENT.md are included beside app.py.
- Deployment limitations are stated: hosted availability, future drift, bias, causality, and untested security remain separate concerns.

A reproducible AI decision system can be rebuilt from source and verified in a fresh process

SESSION 3

Made the selected model inspectable.

SESSION 4

Made the operational decision controllable.

SESSION 5

Makes the controlled behavior portable, testable, auditable, and deployable.

The algorithm is one component. The product is the controlled, verified chain.